

Week 3 - Class 6: Advanced Pointers and References

Why This Matters

In Class 5, you learned the basics of pointers and references. Now we explore advanced topics: - What const means in C++

- How const applies to pointers and references
- Passing by reference vs pointer in functions
- Multiple indirection (pointer-to-pointer)
- How arrays decay to pointers
- Dangling pointers and references

These features are critical for writing safe and efficient C++ code.

What is const?

const means read-only. Once declared const, a variable cannot be modified.

Example:

```
const int DAYS_IN_WEEK = 7;
DAYS_IN_WEEK = 8; // error
```

Why use const?

1. Safety: prevent accidental changes.
2. Clarity: signals intent.
3. Optimization: compiler may optimize.

Const References

Const reference aliases a variable but does not allow modification.

Example:

```
void printValue(const int& ref){ cout<<ref; }
```

Used to pass large objects without copy, while preventing modification.

Const Pointers

Three forms:

```
const int* p = &x; // pointer to const int (cannot change value)
int* const p = &x; // const pointer (cannot resest, but can change value)
const int* const p = &x; // both rules apply
```

Mnemonic: const before * means pointee is read-only. const after * means pointer itself cannot resest.

Passing by Pointer vs Passing by Reference

By Pointer:

```
void increment(int* p){ if(p) (*p)++; }
Call: increment(&a);
```

By Reference:

```
void increment(int& ref){ ref++; }
Call: increment(a);
```

Rule of thumb: use reference when object must exist, pointer when optional (nullable).

Multiple Indirection (Pointer to Pointer)

Pointers can point to other pointers.

C++14 Fundamentals - Student Edition

Example:

```
int x=10; int* p=&x; int** pp=&p; cout<<**pp; //10
```

Diagram:

x:[10]

p:[&x]->x

pp:[&p]->p->x

Arrays and Pointers (Preview)

Arrays decay to pointers.

Example:

```
int arr[3]={1,2,3}; int* p=arr;
```

```
cout<<*p; //1
```

```
cout<<*(p+1); //2
```

Dangling Pointers and References

Dangling Pointer: pointer to variable that no longer exists.

```
int* p; { int x=5; p=&x; } cout<<*p; // UB
```

Dangling Reference: returning reference to local variable.

```
int& bad(){int x=42; return x;} // UB
```

Be mindful of scope.

Common Mistakes

1. Confusing `const int*` vs `int* const`.
2. Returning reference or pointer to local.
3. Using pointers where references suffice.
4. Forgetting parentheses with indirection.

Exercises

1. Write `void square(int& ref)` that squares its argument.
2. Write `void safeIncrement(int* p)` that increments only if `p!=nullptr`.
3. Create `int`, `pointer`, `pointer-to-pointer`, and print values.
4. Create `const int` and bind `const int&` and `const int*`.
5. Explain in your words when to use pointer vs reference in a function.

Project Tie-In

Pointer Playground Part 2: extend practice with `const` references, `const` pointers, function parameters by pointer vs reference, `pointer-to-pointer`, and observe dangling cases.